RAG-Based Customer Support Assistant

1. INTRODUCTION

The RAG-Based Customer Support Assistant is an AI-powered system designed to automate customer query resolution using document-based knowledge retrieval and language model generation.

Traditional customer support systems rely on manual search or static rule-based chatbots, which are inefficient and slow. This system combines Retrieval-Augmented Generation (RAG) with workflow orchestration using LangGraph to solve these limitations.

The system ensures that responses are:

- Contextually accurate
- Grounded in real documents
- Dynamically generated
- Verified using confidence scoring
- Escalated to humans when necessary

2. PROBLEM STATEMENT

Customer support teams handle large volumes of documents (FAQs, policies, manuals). However, existing systems face limitations:

- Manual searching through documents is time-consuming
- Keyword-based search lacks semantic understanding
- Chatbots provide generic or incorrect responses
- No mechanism for handling uncertain queries
- No escalation mechanism for complex cases

3. OBJECTIVE

The main objective is to design an intelligent assistant that:

- Accepts user queries in natural language
- Retrieves relevant information from PDF documents
- Generates meaningful responses using contextual data

- Ensures response accuracy through confidence scoring
- Escalates queries to human agents when required
- Provides structured workflow execution using LangGraph

4. SYSTEM ARCHITECTURE

The system follows a modular pipeline-based architecture.

High-Level Flow:

User Query → Query Embedding → Vector Database Search → Document Retrieval → Response Generation → Confidence Evaluation → HITL Decision → Final Output

Architecture Components:

1. Document Ingestion Layer: Loads PDF documents using PyPDFLoader and extracts textual data.
2. Chunking Layer: Splits documents into smaller chunks (100 characters with 50 character overlap) to preserve context and improve retrieval accuracy.
3. Embedding Layer: Converts text chunks into vector embeddings using sentence-transformers/all-mpnet-base-v2 (768D vectors) to capture semantic meaning.
4. Vector Database (ChromaDB): Stores embeddings in structured format and enables fast similarity search using cosine similarity metric.
5. Retrieval Layer: Accepts user query, converts to embedding, performs similarity search, and retrieves top-2 relevant chunks.
6. Generation Layer: Uses retrieved context to generate response. Template-based approach concatenates actual policy text without hallucination.
7. Workflow Engine (LangGraph): Controls execution flow with structured nodes:
    - Retrieve Node: Vector similarity search
    - Generate Node: Answer creation with confidence scoring
    - HITL Node: Human escalation handling
    - Output Node: Final response formatting
8. Human-in-the-Loop (HITL) Module: Triggered when confidence < 0.7. Escalates to human agent, accepts manual response, returns verified output.

5. DATA FLOW DESIGN

Input Stage: User enters query in natural language.

Retrieval Stage: Query is embedded and matched against document embeddings. Top-2 chunks retrieved based on cosine similarity scores.

Generation Stage: System generates response using retrieved context. Confidence score calculated: confidence = min(0.95, 0.6 + content_length/1000).

Decision Stage: If confidence >= 0.7, return AI response. If confidence < 0.7, escalate to HITL.

HITL Stage (if triggered): Human agent reviews query and system answer, provides verified response, overrides with confidence = 1.0.

Output Stage: Final response formatted with query, answer text, confidence score, and verification status. Displayed to user.

6. TECHNOLOGY STACK

Programming Language: Python 3.9+

Frameworks:

- LangChain: RAG pipeline and document processing
- LangGraph: Workflow orchestration and state management

Embedding Model: sentence-transformers/all-mpnet-base-v2 (768D vectors)

Vector Database: ChromaDB with cosine similarity metric

Document Processing: PyPDFLoader and CharacterTextSplitter

7. RELIABILITY AND CONTROL

Confidence Scoring: Prevents incorrect responses from reaching users. Low-confidence responses are escalated for human review.

HITL Mechanism: Provides fallback for uncertain queries. Ensures critical queries receive human verification.

Retrieval-Based Grounding: Responses grounded in actual document text. No hallucination risk as answers concatenated from real chunks.

Workflow Control: LangGraph enforces structured execution flow. State management ensures consistency throughout pipeline.

8. CONCLUSION

The RAG-Based Customer Support Assistant successfully integrates retrieval systems and workflow orchestration into a unified AI system. It improves traditional support systems by

providing accurate contextual responses, reducing manual workload, enabling intelligent document-based search, and introducing human fallback mechanisms.

This system demonstrates practical implementation of modern AI architecture suitable for production customer support environments.